20-Hour Plan: Real-Time Trading Dashboard & Option Chain

Goal: build the project from your resume, and understand every buzzword in that bullet well enough to explain it in an interview. You learn the theory, not how to write the code from memory.

How each phase works 1. **Study** (≈40% of the phase): read or watch the topics listed. Stop once you can explain each term in 2–3 sentences. 2. **Build** (≈40%): Claude writes the component, then you run it and look at it in the browser. 3. **Notes** (≈20%): read `notes/NN-*.md`. Each file has: what we built → the concepts → how it maps to the resume → likely interview questions.

#	Phase	Hours	Resume keywords it covers
0	Setup & how a web app works	1.0	Vite, TypeScript, Tailwind, HTML5/CSS3
1	React fundamentals + static Watchlist	2.5	React, components, props, state
2	WebSockets + resilient Binance client	2.0	WebSockets, heartbeat, exponential backoff, resubscription
3	Global state + 60 FPS batching	2.0	Zustand (vs Redux Toolkit), requestAnimationFrame, live watchlist
4	Candlestick chart	1.5	TradingView Lightweight Charts, REST APIs, async/await
5	Order screen + positions with live P&L	2.5	Forms, Market/Limit, validation, P&L
6	Order book	1.5	Incremental order book, sorted maps, sequence-gap resync
7	Option chain	2.5	OI, IV, Black–Scholes Greeks, react-window virtualization, memoization
8	Responsive dark UI + Figma	0.5	Responsive design, dark mode, Figma
9	Testing	2.0	Vitest, React Testing Library, Playwright
10	Git, GitHub Actions CI, deploy	1.0	Git, GitHub Actions (CI/CD)
11	Interview revision	1.0	Everything in the resume bullet
	Total	**20.0**	

Phase details

Phase 0: Setup (1h) ✅ built

- **Study:** what HTML, CSS and JS each do · what Node.js and npm are · what a bundler or dev server is (Vite) · why TypeScript (types, interfaces) · what utility-first CSS is (Tailwind)
- **Build:** Vite + React + TS project, Tailwind, a dark app shell with 3 empty panels

- **Notes:** `notes/00-setup.md`

Phase 1: React fundamentals (2.5h)

- **Study:** components & JSX · props · `useState` · re-rendering (what triggers it) · rendering lists & `key` · `useEffect` (basic idea) · component composition
- **Build:** layout components, a static Watchlist from mock data, clicking a symbol selects it
- **Notes:** `notes/01-react-basics.md`

Phase 2: WebSockets (2h)

- **Study:** HTTP vs WebSocket · how the connection opens and closes · Binance WebSocket streams (just read the "Websocket Market Streams" docs page) · heartbeat/ping-pong · exponential backoff with jitter · custom hooks · `useEffect` cleanup
- **Build:** a `BinanceSocket` class that handles heartbeat, reconnect and resubscribe, plus a connection-status badge
- **Notes:** `notes/02-websockets.md`

Phase 3: State + performance (2h)

- **Study:** prop drilling, and why global state solves it · Zustand basics · Redux Toolkit idea (store, slice, action, reducer) to compare · the browser render loop, 60 FPS = 16.6 ms · `requestAnimationFrame` · selectors & avoiding re-renders
- **Build:** a market store, ticks batched per frame, a live watchlist with green/red flashes
- **Notes:** `notes/03-state-and-batching.md`

Phase 4: Charts (1.5h)

- **Study:** OHLC candles & intervals · REST vs WebSocket (history vs live) · `fetch`, Promises, async/await · `useRef` (why charts need direct DOM access)
- **Build:** a candlestick chart that loads history over REST and updates live, with an interval switcher
- **Notes:** `notes/04-charts.md`

Phase 5: Orders & positions (2.5h)

- **Study:** Market vs Limit orders · controlled inputs & forms · validation (tick size, min quantity, balance) · positions, average price, realized vs unrealized P&L
- **Build:** an order panel (simulated, since a real exchange needs API keys), a positions table with live P&L, and pure functions for the P&L math
- **Notes:** `notes/05-orders-pnl.md`

Phase 6: Order book (1.5h)

- **Study:** bids/asks, spread, depth · snapshot + diff updates · sequence numbers & gap detection · why sorted maps

- **Build:** a live order book that follows Binance's documented "manage a local order book" algorithm
- **Notes:** `notes/06-order-book.md`

Phase 7: Option chain (2.5h)

- **Study:** calls/puts, strike, expiry, ITM/ATM/OTM · Open Interest · Implied Volatility · Black–Scholes intuition (no derivation) · Greeks: Delta, Gamma, Theta, Vega · virtualization, and why 200 rows × 20 columns is slow · `React.memo`, `useMemo`
- **Build:** a 200+ strike chain priced live off the spot price with Black–Scholes, rendered through react-window
- **Notes:** `notes/07-option-chain.md`

Phase 8: Responsive + dark UI (0.5h)

- **Study:** mobile-first breakpoints · CSS grid/flex · design tokens · Figma basics (frames, auto-layout, inspect panel)
- **Build:** a mobile layout with tabs and polish
- **Notes:** `notes/08-responsive-ui.md`

Phase 9: Testing (2h)

- **Study:** the testing pyramid (unit / integration / E2E) · Vitest · React Testing Library ("test like a user") · Playwright (real browser)
- **Build:** unit tests for P&L and validation, RTL tests for the order form, a Playwright test of the order flow
- **Notes:** `notes/09-testing.md`

Phase 10: CI/CD (1h)

- **Study:** git basics (commit, branch, push) · what CI is · GitHub Actions workflow YAML · deploying static sites (Vercel/Netlify)
- **Build:** a git repo, a workflow that runs lint + tests + Playwright on every push, and a deploy
- **Notes:** `notes/10-ci-cd.md`

Phase 11: Interview revision (1h)

- Read `notes/INTERVIEW.md`: every resume phrase with a 30-second explanation and follow-up questions.

Honest notes about the resume bullet

- **Orders are simulated.** Binance needs authenticated API keys to place real orders. "Order screen with validation" is accurate. Don't claim you placed real orders.

- **Option chain data:** strikes and Greeks are computed with Black–Scholes from the live spot price. IV/OI are modelled, not taken from an exchange. Say this if you're asked.